

DiceKriging vs RobustGaSP

2D test example

Using the branin function

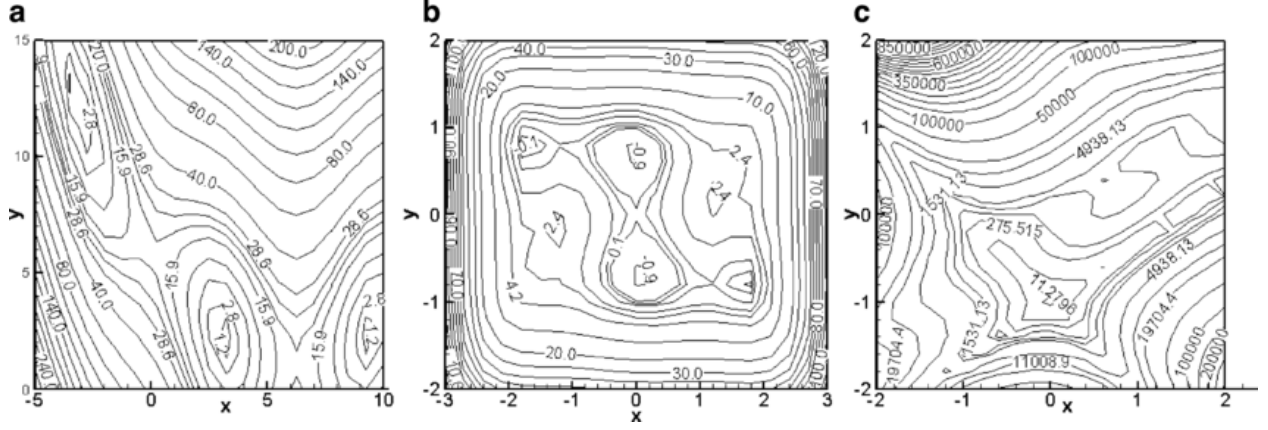


Figure 1: Fig. 2 from Ensemble of surrogates - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Contour-plots-of-two-variable-test-functions-a-Branin-Hoo-function-b-Camelback_fig9_227093096 (accessed 4 Dec 2025). Contour plots of two variable test functions. (a) Branin-Hoo function, (b) Camelback function, (c) Goldstein-Price

We have a 2-dimensional input, one output, and 20 noise-free observations, obtained using the branin function:

$x = \{0.82, 0.7, 0.17, 0.43, 0.55, 0.36, 0.75, 0.11, 0.59, 0.87, 0.26, 0.31, 0.94, 0.71, 0.49, 0.24, 0.06, 0.99, 0.01, 0.63\}$

$x = \{0.67, 0.51, 0.07, 0.4, 0.34, 0.73, 0.36, 0.29, 0.77, 0.56, 0.91, 0.15, 0.84, 0.13, 0.96, 0.48, 0.87, 0.21, 0.04, 0.65\}$

$y = \{95.06, 59.82, 94.35, 15.7, 8.82, 50.1, 39.28, 74.85, 97.35, 56.71, 49.41, 36.75, 113.85, 18.87, 132.65, 13.41, 6.58, 1.48, 263.67, 77.1\}$

Going forward, we will assume a linear mean function of the form

$$m(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2.$$

In RobustGaSP's `rgasp()` function, this involves specifying the trend matrix H :

```
##      [,1]      [,2]      [,3]
## [1,]    1 0.81927243 0.67120228
## [2,]    1 0.69961294 0.50934757
## [3,]    1 0.17314856 0.06616293
## [4,]    1 0.42807701 0.40398944
## [5,]    1 0.54735938 0.33957648
## [6,]    1 0.35877280 0.73307922
```

```
## [7,] 1 0.75380568 0.36245847
## [8,] 1 0.10599233 0.28610379
## [9,] 1 0.58878777 0.76636543
## [10,] 1 0.87422138 0.56243451
## [11,] 1 0.26491909 0.90899509
## [12,] 1 0.31229737 0.15450160
## [13,] 1 0.93614656 0.83886022
## [14,] 1 0.71391978 0.12927032
## [15,] 1 0.48513098 0.95902234
## [16,] 1 0.24309211 0.48177174
## [17,] 1 0.06290104 0.86521487
## [18,] 1 0.98998771 0.20555371
## [19,] 1 0.01390233 0.04203154
## [20,] 1 0.63403032 0.64890147
```

as well as setting `zero.mean` equal to "No". The Gauss (squared exponential) *and* Matern 5/2 kernels are compared for both packages, using the same four functions created in **DiceKriging vs RobustGaSP: 1D test example**.

The output from the two RobustGaSP functions is shown below (first for the Gauss kernel and second for the Matern 5/2).

```
## The upper bounds of the range parameters are Inf Inf
## The initial values of range parameters are 0.006148649 0.005698601
## Start of the optimization 1 :
## The number of iterations is 21
## The value of the marginal posterior function is -72.56137
## Optimized range parameters are 0.4237544 2.161635
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.1952171 0.1833982
## Start of the optimization 2 :
## The number of iterations is 30
## The value of the marginal posterior function is -72.56137
## Optimized range parameters are 0.4237545 2.161636
## Optimized nugget parameter is 0
## Convergence: FALSE
```

```
## The upper bounds of the range parameters are Inf Inf
## The initial values of range parameters are 0.006148649 0.005698601
## Start of the optimization 1 :
## The number of iterations is 32
## The value of the marginal posterior function is -98.50445
## Optimized range parameters are 0.0388389 0.2861011
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.1952171 0.1833982
## Start of the optimization 2 :
## The number of iterations is 17
## The value of the marginal posterior function is -69.24155
## Optimized range parameters are 2.322582 7.703823
## Optimized nugget parameter is 0
## Convergence: TRUE
```

Two checks are carried out. Firstly, the mean width of the 95% CI returned by each of the four set-ups is calculated. Note that I'm getting matching GPs when using **DiceKriging**.

Table 1: Mean width of 95% CI

Package	Kernel	Mean width of 95% CI
DiceKriging	Gauss	5.0385
DiceKriging	Matern 5/2	13.1194
RobustGaSP	Gauss	5.4549
RobustGaSP	Matern 5/2	6.3082

Leave-one-out validation for each of the four set-ups produces the results shown in Tables 2-5 below.

Table 2: DiceKriging with the Gauss kernel (85% validation)

	actual	loo_prediction	lower95	upper95	validated
1	95.06	95.23	88.32	102.14	1
3	94.35	91.13	75.64	106.61	1
4	15.70	13.60	11.72	15.48	0
5	8.82	11.44	9.32	13.57	0
6	50.10	53.27	50.05	56.49	1
7	39.28	39.82	37.94	41.71	1
8	74.85	83.40	72.47	94.33	1
9	97.35	97.58	96.76	98.40	1
10	56.71	58.01	52.29	63.74	1
11	49.41	41.52	30.56	52.47	1
12	36.75	40.96	33.73	48.19	1
13	113.85	100.77	72.61	128.92	1
14	18.87	61.23	-52.52	174.99	1
15	132.65	50.97	-59.62	161.56	1
16	13.41	8.55	1.06	16.05	1
17	6.58	19.13	-27.58	65.83	1
18	1.48	-20.33	-67.61	26.95	1
19	263.67	140.84	101.94	179.75	0
20	77.55	77.46	76.91	78.01	1

Table 3: DiceKriging with the Matern 5/2 kernel (95% validation)

	actual	loo_prediction	lower95	upper95	validated
1	95.06	90.67	80.35	100.99	1
3	94.35	108.33	87.82	128.84	1
4	15.70	15.94	7.20	24.68	1
5	8.82	9.10	-1.84	20.04	1
6	50.10	52.51	40.75	64.27	1
7	39.28	38.72	32.12	45.31	1
8	74.85	84.27	67.23	101.32	1
9	97.35	98.51	94.07	102.94	1
10	56.71	64.89	53.58	76.20	1
11	49.41	48.00	31.81	64.18	1
12	36.75	33.33	14.69	51.96	1

	actual	loo_prediction	lower95	upper95	validated
13	113.85	105.13	68.04	142.22	1
14	18.87	22.30	0.57	44.03	1
15	132.65	126.80	111.24	142.36	1
16	13.41	6.34	-7.46	20.14	1
17	6.58	31.66	-18.21	81.54	1
18	1.48	-25.23	-74.84	24.39	1
19	263.67	166.39	129.50	203.27	0
20	77.55	77.15	73.93	80.37	1

Table 4: RobustGaSP with the Gauss kernel (80% validation)

	actual	loo_prediction	lower95	upper95	validated
1	95.06	97.18	90.28	104.07	1
3	94.35	89.85	73.51	106.18	1
4	15.70	13.44	11.68	15.20	0
5	8.82	11.75	10.05	13.46	0
6	50.10	53.10	49.87	56.34	1
7	39.28	40.18	38.31	42.04	1
8	74.85	79.51	67.50	91.51	1
9	97.35	97.62	96.86	98.37	1
10	56.71	55.52	50.19	60.85	1
11	49.41	39.51	28.02	51.01	1
12	36.75	41.23	34.02	48.43	1
13	113.85	111.14	79.03	143.25	1
14	18.87	14.18	10.17	18.18	0
15	132.65	130.81	127.36	134.26	1
16	13.41	10.90	3.42	18.38	1
17	6.58	23.10	-34.05	80.24	1
18	1.48	-9.03	-66.86	48.80	1
19	263.67	197.15	147.99	246.31	0
20	77.55	77.43	76.91	77.95	1

Table 5: RobustGaSP with the Matern 5/2 kernel (100% validation)

	actual	loo_prediction	lower95	upper95	validated
1	95.06	92.61	87.03	98.19	1
3	94.35	100.11	86.81	113.41	1
4	15.70	15.36	10.79	19.94	1
5	8.82	8.76	3.00	14.51	1
6	50.10	50.77	44.93	56.61	1
7	39.28	39.88	36.27	43.50	1
8	74.85	72.65	61.77	83.52	1
9	97.35	97.66	94.93	100.38	1
10	56.71	61.03	52.70	69.35	1
11	49.41	47.26	40.31	54.20	1
12	36.75	35.57	23.39	47.75	1
13	113.85	119.29	97.64	140.94	1
14	18.87	18.33	5.34	31.32	1

	actual	loo_prediction	lower95	upper95	validated
15	132.65	132.79	124.60	140.99	1
16	13.41	14.43	6.16	22.70	1
17	6.58	22.47	-2.55	47.48	1
18	1.48	-21.88	-58.43	14.66	1
19	263.67	221.31	176.73	265.89	1
20	77.55	77.51	75.18	79.84	1

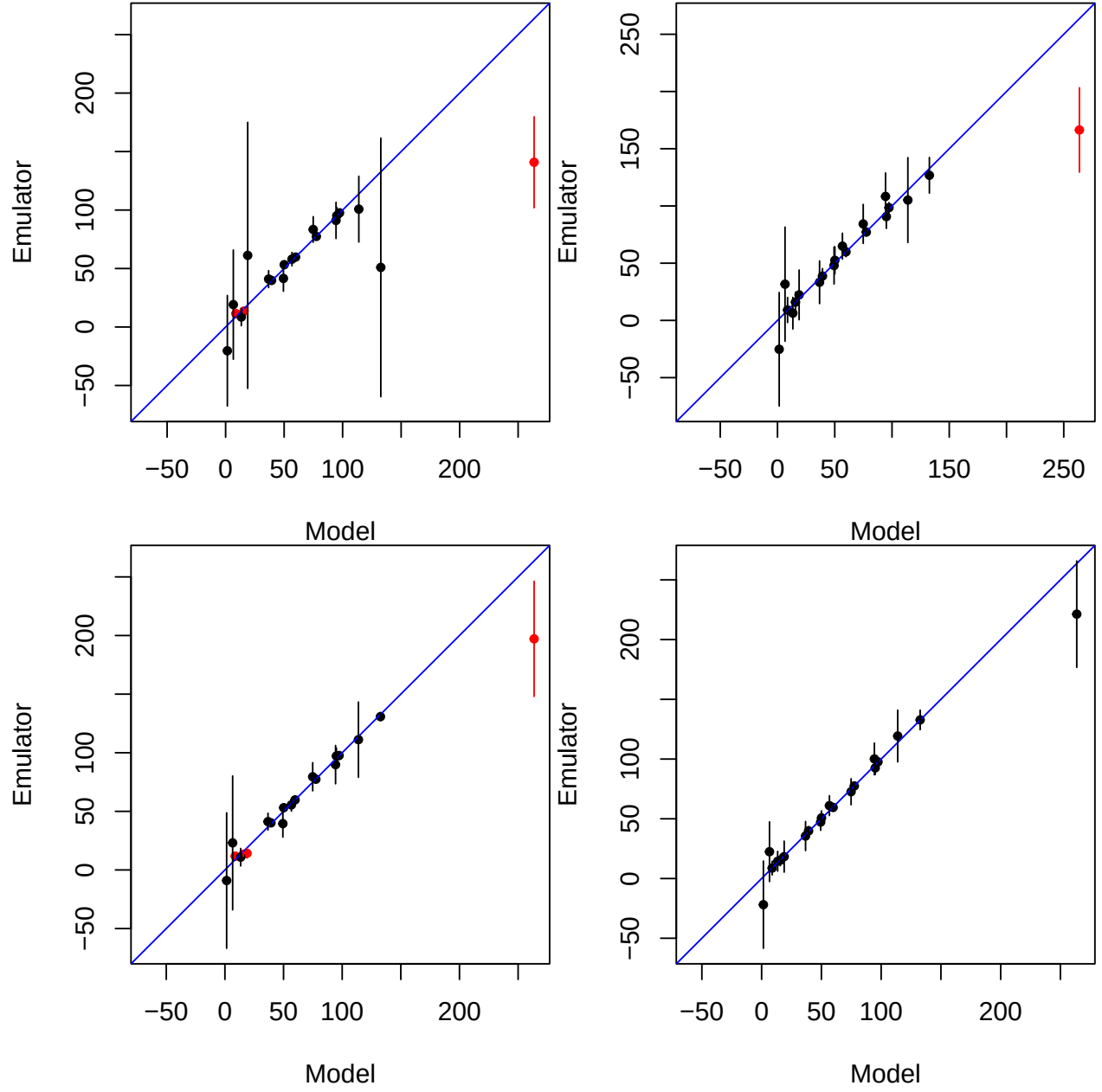


Figure 2: XXXXXXXX.

Going forward, for now, only the Gauss kernel is dealt with; Matern 5/2 will be returned to once I've figured out the estimates returned by the two packages and the derivative of the Matern 5/2 kernel.

The estimates returned by both packages using the Gauss kernel are shown in Table 6.

Table 6: Estimates table

Package	Kernel	Trend 1	Trend 2	Trend 3	Range 1	Range 2	Variance
DiceKriging	Gauss	480.59	-214.54	46.87	0.29	1.32	65247.87
RobustGaSP	Gauss	554.95	-203.61	47.16	0.3	1.53	121693.74

Verifying the estimates by-hand for both packages

To calculate the posterior predictions at the 1000 testing points by-hand, we require the matrix H containing the inputs for the 20 observations and well as matrix A , its inverse, and $\mathbf{t}(x^*)$ for all 1000 of the new inputs x^* . A and $\mathbf{t}(x^*)$ are built using the estimate of the range parameter returned by each package.

For both we calculate

$$\frac{\sum_{i=1}^{1000} \left| (\text{predictions from package})_i - (\text{predictions by-hand})_i \right|}{1000}$$

then divide this by 10^{-8} (machine precision) to get 7 for **DiceKriging** and 20 for **RobustGaSP** (the latter approximately a factor of 2 larger than the former). By-hand predictions therefore match those returned by **DiceKriging** and **RobustGaSP**.